Report: Quantum Simulation Test

Brian Brueggert

October 2025

Abstract

The purpose of this paper is to provide a report, as specified in the Quantum Simulation Test Assignment, proposed by 'I Build Cool Stuff', in the MayoD-LTD/quantum-dev-hiring-test GiitHub repository, namely; to provide a circuit workflow explanation, results of a requested amplitude calculation, and performance benchmarks and hardware requirements. This report also contains a list of the prerequisites libraries in the Requirements (Section 6) at the end.

1 Circuit Workflow Explanation

1.1 Original Circuit Workflow Explanation

We are asked to review the /circuit_generation workflow with the following structure: Original Directory Structure.

We find the /circuit_gen directory, where requirement.md contains to specific instructions for this test, and peaked_circuit_diff=37.qasm is a 37-qubit peaked quantum circuit in OPENQASM 2.0 format. .gitmodules hosts files to coordinate the repository structure.

The default entry point is /circuit_gen/main.py. The directories with the __init__ .py files allow the diretories to be treated as packages, assisting with package initialization and defining the public interface. First, main.py determines whether GPU ('cuda') is enabled, or uses the default CPU. Several parameters are set (diff_level, seed, etc.), and from the files within /circuit_gen/lib/, generates a peaked quantum circuit using the set parameters (i.e. specified difficulty, given seed).

/circuit_gen/main.py dynamically loads /circuit_gen/lib/circuit_gen.py, which imports the CircuitParams class (build peaked circuit with given parameters) and DEVICE string (CPU/GPU 'cuda'). It also contains a greedy hyper-optimizer, a random nxn unitary matrix generator function for GPU's, another to randomly apply gates to the tensor network on the GPU, and normalization and loss functions. Here we will also find the TNModel class, serving to wrap quimb tensor networks for optimization, and prepare_model_for_seed, which builds the tensor networks (both inital and TNModel) given a seed. make_qmps function creates a Matrix Product State (MPS) from the tensor network on a GPU. Other functions create a temporary environment, build the 'unis', generate obfuscated variants from the tensors and make a circuit utilizing a lucky seed.

/circuit_gen/lib/circuit_gen.py, in turn, also calls /circuit_gen/lib/circuit.py, which contains class definitions of quantum logic gates, including a basic single unitary, Hadamard, S, $S^\dagger$, RX, RY, RZ, RXX, RYY, RZZ, CNOT, U3, SU4, SU4Decomp, IsingDecomp, CnotsDecomp, as well as a PeakedCircuit class for generating the peaked circuit, saved in OPENQASM 2.0 format, from the gates.

base_cache.py, obfuscate.py and verification.py serve roles readily evident in their names. The /decompose directory holds cnots.py and ising.py, for respective decomposition of tensor networks.

There is another entry point within /circuit_gen/prototyping/su4_grad.py. It shows a hard-coded reference used to generate su4_grad_cnots.txt, and the assumption is that in the past it had been used to produce su4_grad_ising.txt.

Also, other entry points exists, such as /circuit_gen/peaked_circuits/peaked_circuits.ipynb (PQC optimization from RQC example), /circuit_gen/peaked_circuits/torch_example.ipynb (TNModel with greedy hyper-optimizer to make peaked circuit example), /circuit_gen/peaked_circuits/circuits.py and /circuit_gen/peaked_circuits/functions.py. There are two pickle files, pqc_example.pkl and rqc_example.pkl, which are n-dimensional arrays in serialized binary files. Entropy.pdf is a graph of bipartite entropy as a function of circuit depth, and peak.pdf is a graph of avergage peak weight s a function of n. Also in this directory, is 'On verifiable quantum advantage with peaked circuit sampling' (arxiv:2404.14493: Aaronson and Zhang, 2024).

The final existing entry point to discuss is /circuit_gen/quimb_cache_utils.py. This can be run to clear the local quimb Python library cache.

1.1.1 main.py Code Implementation

```
(<venv>) /quantum-dev-hiring-test/circuit_gen/$ sudo python3.12 main.py
```

There were a couple of issues encountered while working to get main.py running. First, main.py imports from˜/lib/circuit_gen.py, which references a non-existent directory, peaked_circuit_creation.lib.circuit. With the provided structure, this was changed in lines 11, 14 and 17 by omitting the 'peaked_circuit_creation.' prefix from the directory reference. Second, the same issue is found in˜/lib/decompose/__init__.py, and was modified in the same manner with respect to lines six and seven.

With these modifications, main.py ran with the default parameter settings, returning:

Qubits: 35
rqc depth: 18
pqc depth: 7

This is indicative of a 35-qubit peaked circuit with a random quantum circuit depth of 18 and a parametric quantum circuit depth of seven, which the program intends to create. After several more hours of running, the system experienced OOM issues, and the PID required force killing. It should be noted that at this time, the system consists of a 2011 MacBook Pro, 2.4 GHz Intel i5 CPU, 8 GB 1333 MHz DDR3 RAM and an Intel HD Graphics 3000 512 MB GPU, running macOS High Sierra 10.13.16. It lacks PyTorch GPU acceleration, since it's missing the necessary Apple silicon (M1/M2/M3/M4) and modern AMD GPU's required for Apple's Metal Performance Shaders (MPS, disambiguation) framework. Ideally, given a week or so of lead time, a 2022 MacBook Pro M1 Max, 10-core CPU, 32-core GPU, 16-core TPU, with 64 GB RAM (Unified Memory, 400 GB/s) running macOS 26 Tahoe could and would be configured as the system on which to run for testing/development, at least in the initial stages. A CPU/GPU cloud-based subscription environment would allow for tiered system scaling ability, as well as the ability to take advantage of the Directed Acyclic Graph (DAG) structure inherent in Spark software architecture, for example, allowing for optimization of parallelization (or performing all commutative operations in memory), as well as separation of storage and compute resources. Unfortunately, such a lead time was viewed as infeasible at the time of this report.

Next, the diff_level parameter was set to zero, and main.py was run again. We can see from the static method function from_difficulty, defined in the CircuitParams class in /circuit_gen/lib/circuit_gen.py:

```
def from_difficulty(level: float):
    safe = max(level + 3.9, 0.1)
    nqubits = int(round(12 + 10 * np.log2(safe)))
    nqubits = max(10, nqubits)
    rqc_mul = 150 * np.exp(-nqubits / 4) + 0.5
    rqc_depth = round(rqc_mul * nqubits)
    pqc_depth = max(1, nqubits // 5)
```

return CircuitParams(level, nqubits, rqc_depth, pqc_depth) ,

that this should return:

Qubits: 32
rqc depth: 18
pqc depth: 6 ,

as it does. Therefore, to create a 37-qubit peaked circuit using main.py, one would need to set the diff_level parameter to 1.757-2.162 (i.e. 2.0). (As an aside, the seed value, mentioned earlier, is used to force a non-deterministic random number generator to behave deterministically, with respect to the given seed value.) If creating a 35-qubit peaked circuit exceeded the 8 GB RAM on the system, then creating a 37-qubit peaked circuit on this system should surely suffer the same fate. However, after several hours of high memory pressure, the 32-qubit peaked circuit creation program failed with:

/circuit_gen/lib/circuit.py (ln. 632):
def _map_with_bool() can't get local object PeakedCircuit from su4_series.⟨locals⟩._pool_initializer

It is at this point, due to time constraints, that attention is diverted away from making main.py perform on this system, and invested in creating simulate_amplitude.py.

1.2 simulate_amplitude.py Workflow Explanation

The requested Python code file, simulate_amplitude.py, has been developed to run from /circuit_gen/simulate_amplitude.py. However, depending on the use case, it could belong under /circuit_gen/lib/, /circuit_gen/peaked_circuits/ or /circuit_gen/prototyping/, and could easily be revised accordingly. The initial thought for finding the peak state and probability amplitude of a 37-qubit peaked circuit, especially on this system, would be to use an MPS-based algorithm, previously developed by the author. However, the instructions request using tensor network sampling/contraction, and tensor network is differentiated from MPS in the given data structures. Therefore, a much less robust, and less ideal on this system, tensor network solution is presented. (The MPS version (Algorithm 2) is included as an afterthought, since the TN version failed to complete in a timely manner on this system.)

1.2.1 simulate_amplitude.py Code Implementation

```
(<venv>) /quantum-dev-hiring-test/circuit_gen/$ sudo python3.12 simulate_amplitude.py
```

This code takes the revised 37-qubit 'peaked_circuit_diff=37_2.qasm' file as input, builds a tensor network out of it which is representative of the wavefunction Ψ, and uses an adaptive cotengra optimized contraction strategy, if available, otherwise falls back on a greedy contraction path strategy. This is an attempt at balancing cost versus memory, dynamically selecting among the available contraction tree methods, favoring those which are more efficient. The number of samples is set to 500, and the most frequent bitstring is recorded and returned. This code ran for ∼48 hours on the current system without memory issues, but hadn't converged on a solution yet, so the process was terminated by the user, due to time constraints. Given enough time, in the past the algorithm has been able to process peaked circuits with more qubits, although it is unknown what the RQC and PQC depths of those circuits were (they may have been shallower). In any case, as an afterthought, an MPS-based method (Algorithm 2) is explored next.

1.2.2 Algorithm: Hidden Bitstring Extraction via Tensor Network Simulation

Algorithm 1 Tensor Network Quantum Bitstring Sampling

Require: QASM circuit file `path`, number of samples N_s, maximum bond dimension χ, contraction method $\mathscr{C}$ ("auto": "cotengra.auto" adaptive cotengra optimized if installed, otherwise "greedy")

Ensure: Set of measured bitstrings $\{\tilde{x}^{(k)}\}_{k=1}^{N_s}$ distributed according to $P(x) = |\langle x|\Psi\rangle|^2$

Initialize quantum circuit as tensor network

$\mathscr{N} \leftarrow$ LoadTensorNetwork(`path`) ▷ Convert QASM circuit to quimb tensor network $\mathscr{N}$ representing $|\Psi\rangle$

Simplify and compress internal structure of $\mathscr{N}$ to limit index growth

$$\mathscr{N} \leftarrow \text{Simplify}(\mathscr{N}, = \chi)$$

Set contraction strategy $\mathscr{C}$ and initialize random seed

for $k = 1$ **to** N_s **do** [1em] Sequentially collapse each open qubit index by conditional sampling:

for $i = 1$ to n **do**

    Compute marginal probabilities for qubit i given fixed earlier bits:

$$p_{i,b} = \|\text{ContractPartial}(\mathscr{N},\, x_{<i},\, b)\|^2$$

    Normalize $p_{i,0}$ and $p_{i,1}$ to unity

    Sample outcome $x_i \sim \{0, 1\}$ according to $p_{i,b}$

    Project tensor index i to fixed value x_i:

$$\mathscr{N} \leftarrow \mathscr{N}\big|_{\sigma_i = x_i}$$

end for

Record sampled bitstring $\tilde{x}^{(k)} = (x_1, x_2, \ldots, x_n)$

return $\{\tilde{x}^{(k)}\}_{k=1}^{N_s}$

1.3 simulate_amplitude_mps_src.py Workflow Explanation

1.3.1 simulate_amplitude_mps_src.py Code Implementation, 4-qubits

```
(<venv>) /quantum-dev-hiring-test/circuit_gen/$ sudo python3.12
    simulate_amplitude_mps_src.py --nqubits 4
```

Before running this code, check to ensure the path is set to the appropriate .qasm file, "../P1_little_peak.qasm" in this case (with 4 qubits), around line 145. Save the file, and ensure referencing the '–nqubits' input parameter of '4' when running it.

Output: (almost immediate)

PyTorch GPU not available. Using NumPy backend.
Final sampled outcomes (top 5): 1001: 0.6693458951415708
Most likely outcome (peak): 1001: 0.6693458951415708

1.3.2 simulate_amplitude_mps_src.py Code Implementation, 28-qubits

```
(<venv>) /quantum-dev-hiring-test/circuit_gen/$ sudo python3.12
    simulate_amplitude_mps_src.py --nqubits 28
```

Before running this code, check to ensure the path is set to the appropriate .qasm file, "../P2_swift_rise.qasm" (with 28 qubits), around line 145. Save the file, and ensure referencing the '–nqubits' input parameter

of '28' when running it.

Output: (a moment)

PyTorch GPU not available. Using NumPy backend.

Final sampled outcomes (top 5):
1100101101100011011000011100: $1.0125092390755827e-10$

Most likely outcome (peak):
1100101101100011011000011100: $1.0125092390755827e-10$

1.3.3 simulate_amplitude_mps_src.py Code Implementation, 37-qubits

```
(<venv>) /quantum-dev-hiring-test/circuit_gen/$ sudo python3.12
    simulate_amplitude_mps_src.py --nqubits 37
```

Before running this code, check to ensure the path is set to the appropriate .qasm file, "../peaked_circuit_diff=37_2.qasm" in this case (the revised 37-qubit .qasm file), around line 145 Save the file, and ensure referencing the '–nqubits' input parameter of '37' when running it.

Output: (a moment)

PyTorch GPU not available. Using NumPy backend.
~/lib/python3.12/site-packages/quimb/tensor/circuit.py:215: SyntaxWarning: Unsupported operation ignored: creg
warnings.warn(

Final sampled outcomes (top 5):
0110111000001010100010011000110101101: $3.207763447468762e-15$
0001000001101100001011110111111110110: $4.9187707979912174e-14$

Most likely outcome (peak):
0110111000001010100010011000110101101: $3.207763447468762e-15$

Up until this point, the maximum bond dimension of the MPS, χ, has been set in lines 46 and 47 to χ=32, which had been sufficient for the previous peaked circuits not only because they had less qubits, but also because they were less complex, with less entanglement. To account for this in the MPS model, we next set χ=33 on both of the aforementioned lines.

Output: (a moment or so)

PyTorch GPU not available. Using NumPy backend.
~/lib/python3.12/site-packages/quimb/tensor/circuit.py:215: SyntaxWarning: Unsupported operation ignored: creg
warnings.warn(

Final sampled outcomes (top 5):
0110101000011110100011011100100101101: $1.222230164319455e-15$
0001000001101100001011110111111110110: $6.961638506580622e-15$

Most likely outcome (peak):
0110101000011110100011011100100101101: $1.222230164319455e-15$

Indeed, it appears that the MPS model had not been capturing enough of the complexity of the peaked circuit depth, so we now set χ=34....

Output: ($\sim$ 15-20 hrs.)

PyTorch GPU not available. Using NumPy backend.
~/lib/python3.12/site-packages/quimb/tensor/circuit.py:215: SyntaxWarning: Unsupported
operation ignored: creg
warnings.warn(

Final sampled outcomes (top 5):
01101110000011101000100111001001011O1: $2.412466250307149e-11$
00010000011011000010111101111111110110: $3.704995783310298e-12$

Most likely outcome (peak):
01101110000011101000100111001001011O1: $2.412466250307149e-11$

simulate_amplitude_mps_src.py is called with an input parameter, '–nqubits', and the associated .qasm file is supplied via hard-coded reference. Utilizing Autoray, it prioritizes PyTorch GPU, if it is available, over NumPy CPU, as a backend. The peaked circuit is transformed into an MPS (χ=32, 33 and 34), representative of the wavefunction Ψ. The marginals of each qubit are computed, and then projected, compressed and normalized. The entropy is recorded, and the final wavefunction Ψ is converted into a bitstring and its probability amplitude is also calculated. In the case of 37-qubits, the code will also return the probability amplitude for a preselected bitstring, namely:

$|00010000011011000010111101111111110110\rangle$, $3.704995783310298e-12$.

1.3.4 Algorithm: Optimized Symbolic Resonance Collapse (MPS)

Algorithm 2 Optimized Symbolic Resonance Collapse (MPS)

Require: QASM file path `path`, mixing rate $\eta \in (0,1)$, max bond χ, cutoff ε, tolerance τ, max cycles T

Ensure: Hidden bitstring $\hat{b} \in \{0,1\}^n$

$|\Psi\rangle \leftarrow \text{LoadMPS}\big(\texttt{path}, = \chi\big)$

for $t = 1 \to T$ **do**

  Compute marginals for each qubit $i = 1, \ldots, n$:

$$p_{i,0} \;=\; \langle\Psi| \left(|0_i\rangle \langle 0_i|\right) |\Psi\rangle\,, \quad p_{i,1} \;=\; \langle\Psi| \left(|1_i\rangle \langle 1_i|\right) |\Psi\rangle$$

  if $\forall\, i : |p_{i,0} - p_{i,1}| \geq \tau$ **then**

    break

  end if

  Define uncertain qubits:

$$\mathcal{Q} \leftarrow \{\, i \mid |p_{i,0} - p_{i,1}| < \tau \}.$$

  for all $i \in \mathcal{Q}$ **do**

    for $b = 0, 1$ **do**

      Copy state: $|\Phi\rangle \leftarrow |\Psi\rangle$

      Project: $P_{i,b} = \frac{1+(-1)^b Z_i}{2}$, then

$$|\Phi\rangle \leftarrow P_{i,b}\, |\Phi\rangle\,.$$

      Truncate:

$$|\Phi\rangle \leftarrow \text{compress}\big(|\Phi\rangle\,,\, \chi,\, \varepsilon\big).$$

      Mix & normalize:

$$|\Psi\rangle \leftarrow \eta\, |\Psi\rangle + (1-\eta)\, |\Phi\rangle\,, \quad |\Psi\rangle \leftarrow \frac{|\Psi\rangle}{\| \,|\Psi\rangle\, \|}.$$

    end for

  end for

  Record entropy:

$$S_t \leftarrow -\sum_{i=1}^{n} \sum_{b=0}^{1} p_{i,b} \log_2\big(p_{i,b}\big).$$

end for

for $i = 1$ **to** n **do**

  $\hat{b}_i \leftarrow \arg\max_{b \in \{0,1\}} p_{i,b}$

end for

return $\hat{b}$

2 Amplitude Calculation Results

The provided peaked circuit .qasm file, peaked_circuit_diff=37.qasm (last modified 10/1/2025), has a measurement on line 13,580:

measure q → c; ,

which is removed, and the file is then saved as peaked_circuit_diff=37_2.qasm. This is the 37-qubit peaked circuit that will be processed by simulate_amplitude.py and simulate_amplitude_mps_src.py, due to the previously noted difficulties in getting main.py to run successfully in order to generate a new 37-qubit peaked circuit, within the (imagined) time frame. Similarly, for our smaller test circuit, we will use the 4-qubit P1_little_peak.qasm file (acquired previously).

The tensor network sampling/contraction method was unable to complete the amplitude calculation within a reasonable amount of time for the purpose of this report. As can be seen from the output of simulate_amplitude_mps_src.py, in Section 1.3.1, the tractable 4-qubit peaked circuit returns a bitstring of $|1001\rangle$ with a probability amplitude of 0.669. Furthermore, with the maximum bond dimension of the MPS model set appropriately (χ=34), we find the results to indicate a peak state at $|011011100000111010001001110010010 1101\rangle$, with proabability amplitude $2.412466250307149e-11$. Here we also find the probability amplitude for the given bitstring, $|000100000110110000101111011111111 0110\rangle$, to be $3.704995783310298e-12$.

3 Performance Benchmarks and Hardware Requirements

Quantum tensor network memory depends on the representation, number of qubits n, and bond dimension χ (virtual/bond indices).

$$\text{memory}_{\text{full tensor network}} \sim \mathcal{O}(2^n) \quad .$$

For a 37-qubit tensor network representation, an array of length 2^n (binary qubits), where $n = 37$, or $\approx 1.37 \times 10^{11}$ amplitudes is required. For complex64 (32-bit real, 32-bit imaginary) type memory, this gives 8 bytes/amplitude times 2^n bytes. (For complex128 (64-bit real, 64-bit imaginary) type memory, we get 16 bytes/amplitude times 2^n bytes.)

For a 37-qubit tensor network, 1.1 TB/2.2 TB (complex64/complex128) of memory is required, which is large in terms of single-GPU VRAM or single-machine RAM. On a typical 24 GB VRAM GPU execution would result in OOM. On an 8 GB RAM CPU, 1.1 TB/2.2 TB (complex64/complex128) memory is required, or needs to be managed, or will result in OOM, as well. In the case of 40-qubits, we get a full tensor network memory requirement of 8 TB/16 TB (complex64/complex128).

Tensor network contractions can avoid $\mathcal{O}(2^n)$ for structured circuits, where peak memory scales as:

$$\text{peak memory}_{\text{tensor network contraction}} \sim \mathcal{O}(2^\omega) \ ,$$

where ω is the effective tensor network tree width. While this may be an achievable for small, structured circuits, for a high $\omega = 30/40$, this results in $\sim$ 8 GB/8 TB (complex64). Therfore, it's possible for the tree width to grow during contractions, especially for highly entangled states, allowing $\omega \to n$, forcing exponential memory.

MPS memory is linear in n and polynomial in χ:

$$\text{MPS memory}_{\text{general}} \sim \mathcal{O}(2n)\mathcal{O}(\chi^2) \quad .$$

Using $2n\chi^2$ as our estimate, for $n = 40$, $\chi = 64$, this gives $\sim$ 2.44 MB (complex64), and with $\chi = 2048$, this grows to $\sim$ 2.5 GB (complex64). Thus, highly entangled circuits have higher χ values, which drives up the memory requirements, though these figures are more reasonable.

In practice, Cuda and other runtime overheads, temporary buffers and extra intermediate tensors consume a significant portion of the VRAM, so allowing an $\sim$ 30% buffer is advisable. Had there been more time to iterate, code level timers and memory logs would be utilized. The peak memory during simulation is what matters most, and is the result of the tensor network representation, intermediate tensors created during contractions, gates, measurements, etc., buffers and temporary copies, and the autograd/optimizer state for gradient based optimizations.

4 Index

List of Algorithms

1 Tensor Network Quantum Bitstring Sampling 4
2 Optimized Symbolic Resonance Collapse (MPS) 7

5 Directory Structure

Original Directory Structure

/quantum − dev − hiring − test
  /circuit_gen
    /lib
      /decompose
        __init__.py
        cnots.py
        ising.py
      __init__.py
      base_cache.py
      circuit.py
      circuit_gen.py
      obfuscate.py
      verification.py
    /peaked_circuits
      README.md
      circuits.py
      entropy.pdf
      functions.py
      peak.pdf
      peaked_circuits.ipynb
      pqc_example.pkl
      requirements.txt
      rqc_example.pkl
      torch_example.ipynb
    /prototyping
      su4_grad.py
      su4_grad_cnots.txt
      su4_grad_ising.txt
    .gitmodules
    __init__.py
    main.py
    quimb_cache_utils.py
  peaked_circuit_diff = 37.qasm
  requirement.md

6 Requirements

Python 3.12.12, as well as the list below; information which is also contained in requirements.txt file included in Project.

Package	Version
alembic	1.17.0
autoray	0.8.0
blueqat	2.0.4
blurhash	1.1.5
boost	0.1
certifi	2025.10.5
charset-normalizer	3.4.4
colorlog	6.9.0
contourpy	1.3.3
cotengra	0.7.5
cycler	0.12.1
cytoolz	1.0.1
decorator	5.2.1
dill	0.4.0
filelock	3.20.0
fonttools	4.60.1
fsspec	2025.9.0
greenlet	3.2.4
idna	3.11
iniconfig	2.1.0
Jinja2	3.1.6
kiwisolver	1.4.9
llvmlite	0.42.0
Mako	1.3.10
MarkupSafe	3.0.3
Mastodon.py	2.1.4
matplotlib	3.8.3
mpmath	1.3.0
networkx	3.5
numba	0.59.1
numpy	1.26.4
$\mathrm{opt}_e insum$	3.4.0
$optuna$	4.5.0
$packaging$	25.0
$pillow$	11.3.0
$pluggy$	1.6.0
$psutil$	7.1.0
$pyparsing$	3.2.5
$pytest$	8.1.1
$python-dateutil$	$2.9.0.post0$
$python-magic$	0.4.27
$pytorch-ranger$	0.1.1
$PyYAML$	6.0.3
$qiskit$	2.2.1
$qiskit-aer$	0.17.2
$qiskit-quimb$	0.0.9
$quimb$	1.11.2

Package	Version
six	1.17.0
SQLAlchemy	2.0.44
stevedore	5.5.0
sympy	1.14.0
toolz	1.0.0
torch	2.2.2
torch-optimizer	0.3.0
tqdm	4.66.2
typing_extensions	4.15.0
urllib3	2.5.0
requests	2.32.5
rustworkx	0.17.1
scipy	1.11.2

7 Bibliography

[1] whisdev. *MayoD-LTD*. GitHub. Last modified October 2025. https://github.com/MayoD-LTD/quantum-dev-hiring-test/tree/main. (Accessed October 15, 2025). MayoD-LTD/quantum-dev-hiring-test

[2] Aaronson, Scott, and Yuxuan Zhang. "On verifiable quantum advantage with peaked circuit sampling." arXiv, May 21, 2024. https://arxiv.org/abs/2404.14493.